NOTE:
Parallel reduction can be achieved with both cpu and gpu.

For cpu based execution use HPC_3.cpp file/code

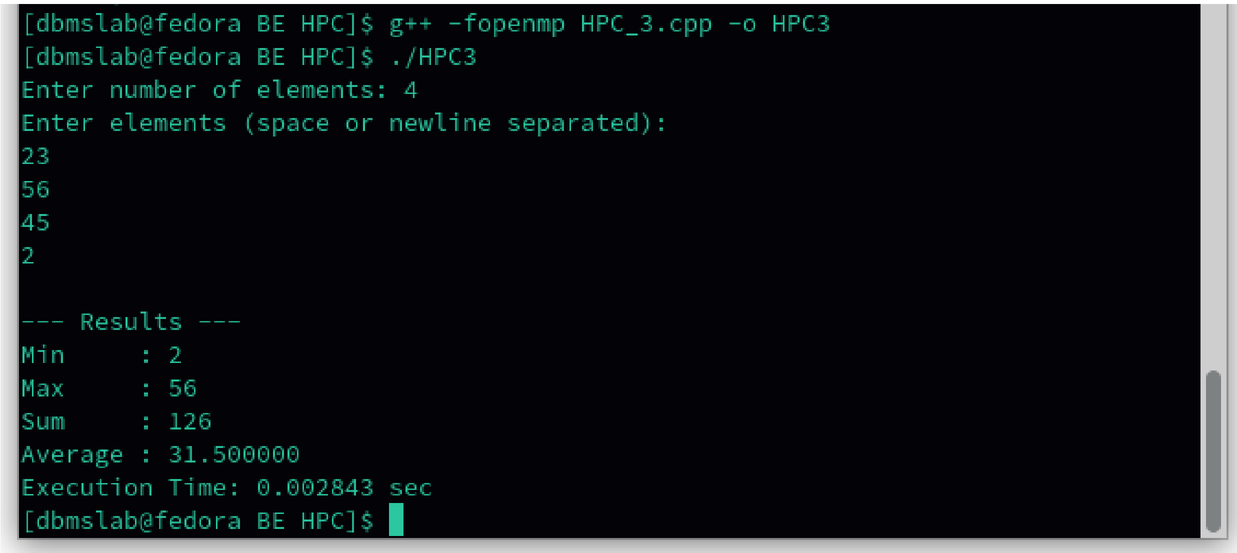
For CUDA based execution use HPC_3_CUDA.cu file/code. For this ensure you have CUDA compatible gpu preferably NVIDIA hardware.

CODE:

```
//this runtime uses CPU execution. Parallel reduction can also be achieved using NVIDIA GPU
and CUDA


#include <iostream>
#include <vector>
#include <omp.h>
#include <iomanip>

using namespace std;

// CPU parallel reduction for Min, Max, Sum, Average
void cpu_reduce(const vector<int> &arr, int &min_val, int &max_val, long long &sum, double
&avg) {
	int n = arr.size();
	min_val = arr[0];
	max_val = arr[0];
	sum = 0;

	#pragma omp parallel for reduction(min:min_val) reduction(max:max_val)
reduction(+:sum)
	for (int i = 0; i < n; i++) {
	if (arr[i] < min_val) min_val = arr[i];
	if (arr[i] > max_val) max_val = arr[i];
	sum += arr[i];
	}

	avg = static_cast<double>(sum) / n;
}

int main() {
	int n;
```

```
    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> arr(n);
    cout << "Enter elements (space or newline separated):\n";
    for (int i = 0; i < n; i++)
    cin >> arr[i];

    int min_val, max_val;
    long long sum;
    double avg;

    double start = omp_get_wtime();

    cpu_reduce(arr, min_val, max_val, sum, avg);

    double end = omp_get_wtime();

    cout << fixed << setprecision(6);
    cout << "\n--- Results ---\n";
    cout << "Min    : " << min_val << "\n";
    cout << "Max    : " << max_val << "\n";
    cout << "Sum    : " << sum << "\n";
    cout << "Average : " << avg << "\n";
    cout << "Execution Time: " << end - start << " sec\n";

    return 0;
}
```

OUTPUT:




```
[dbmslab@fedora BE HPC]$ g++ -fopenmp HPC_3.cpp -o HPC3
[dbmslab@fedora BE HPC]$ ./HPC3
Enter number of elements: 4
Enter elements (space or newline separated):
23
56
45
2

--- Results ---
Min     : 2
Max     : 56
Sum     : 126
Average : 31.500000


Execution Time: 0.002843 sec
[dbmslab@fedora BE HPC]$
```